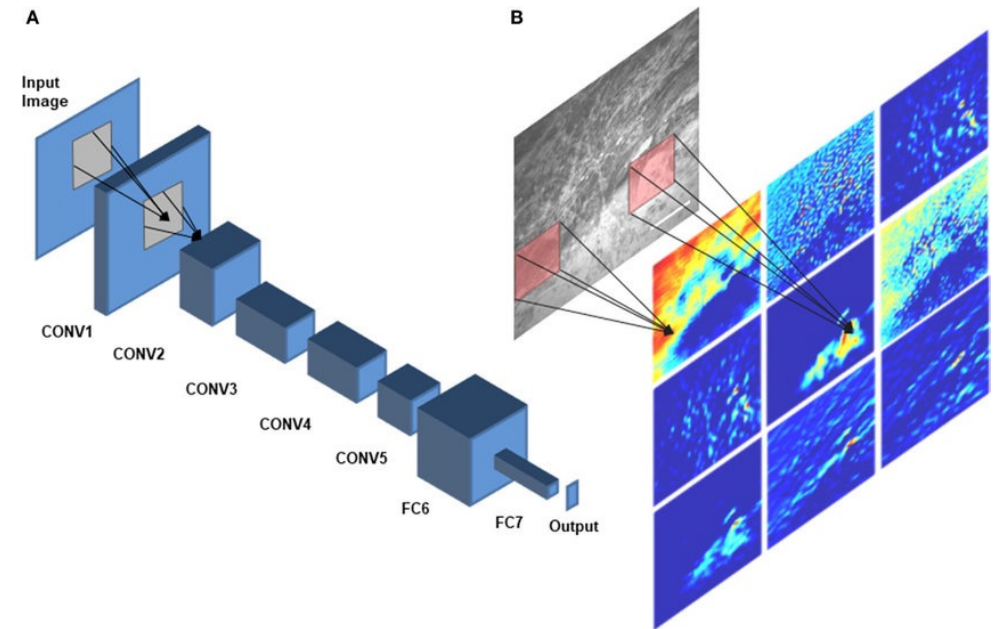


Convolutional Neural Networks for Image Classification II



27. März 2025

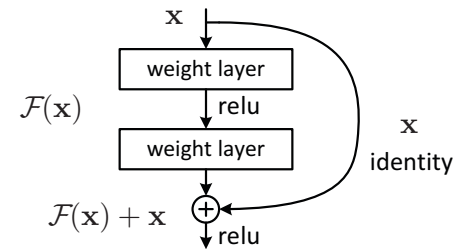
Repetition

- Does the architecture of the network matter?
- What is the 'best' network architecture?
- Do you ever have to build your own network for an application?

Does architecture matter?

Important building blocks:

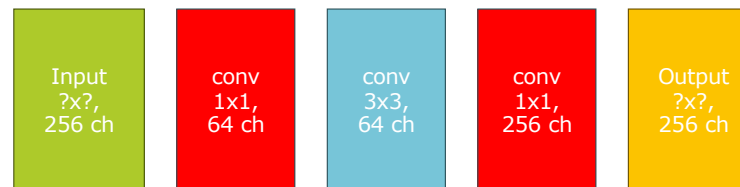
- Skip Connections



- Batch Normalization

$$O_{b,c,x,y} \leftarrow \gamma_c \frac{I_{b,c,x,y} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c \quad \forall b, c, x, y$$

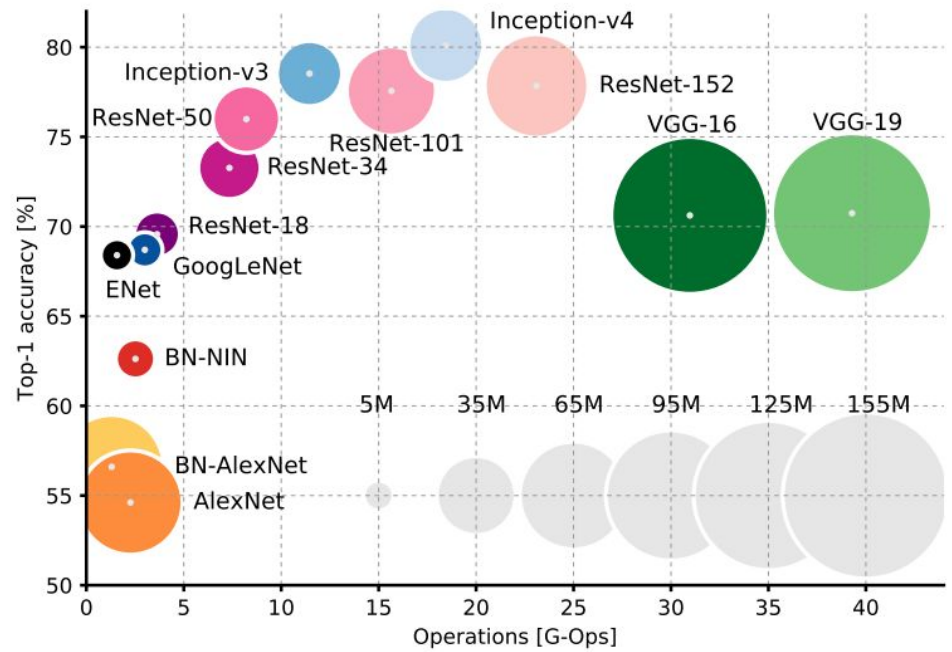
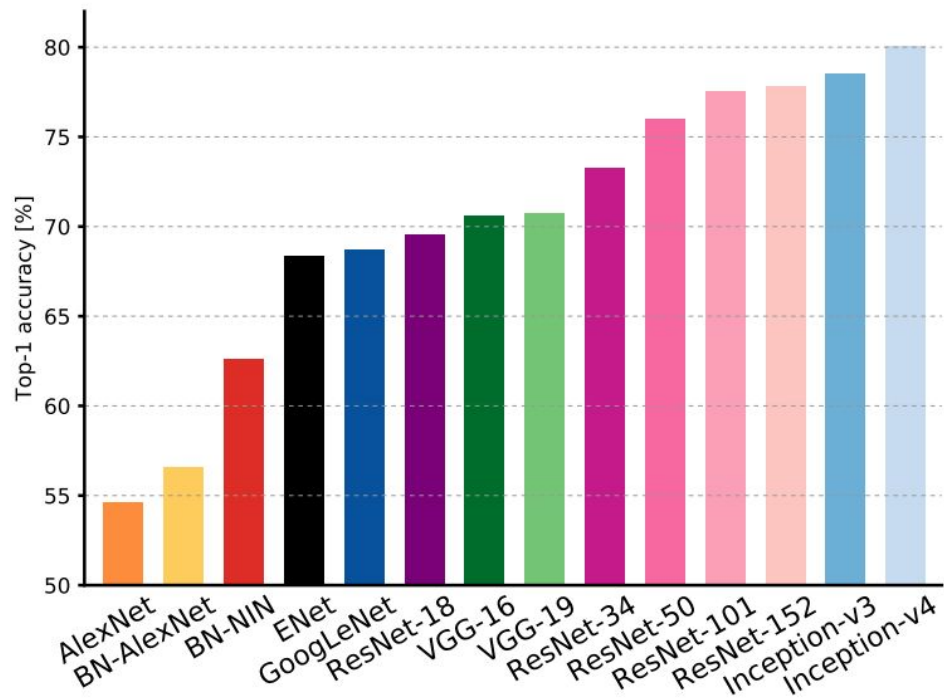
- Bottleneck Layers



Does architecture matter?

Tradeoff between:

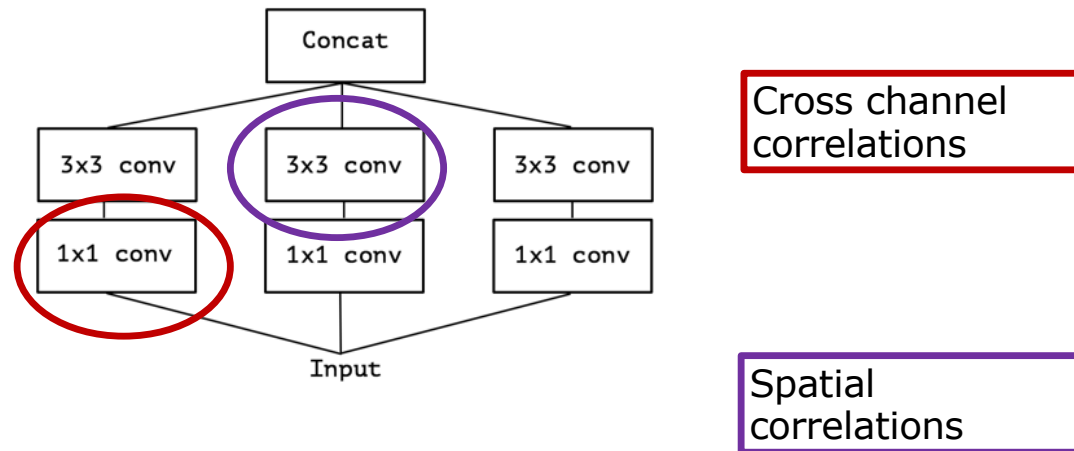
Classification Performance vs. Processing Time vs. Size of network (and needed memory)



Figures copyright Alfredo Canziani, Adam Paszke, Eugenio Culurciello, 2017.

Depthwise seperable filters

- A convolutional filter has correlations across its depth (channel) and in its spatial dimension simultaneously
- Is this necessary or can the two be decoupled?
- Bottleneck layers already have depth-only correlations.



Depthwise separable convolutions

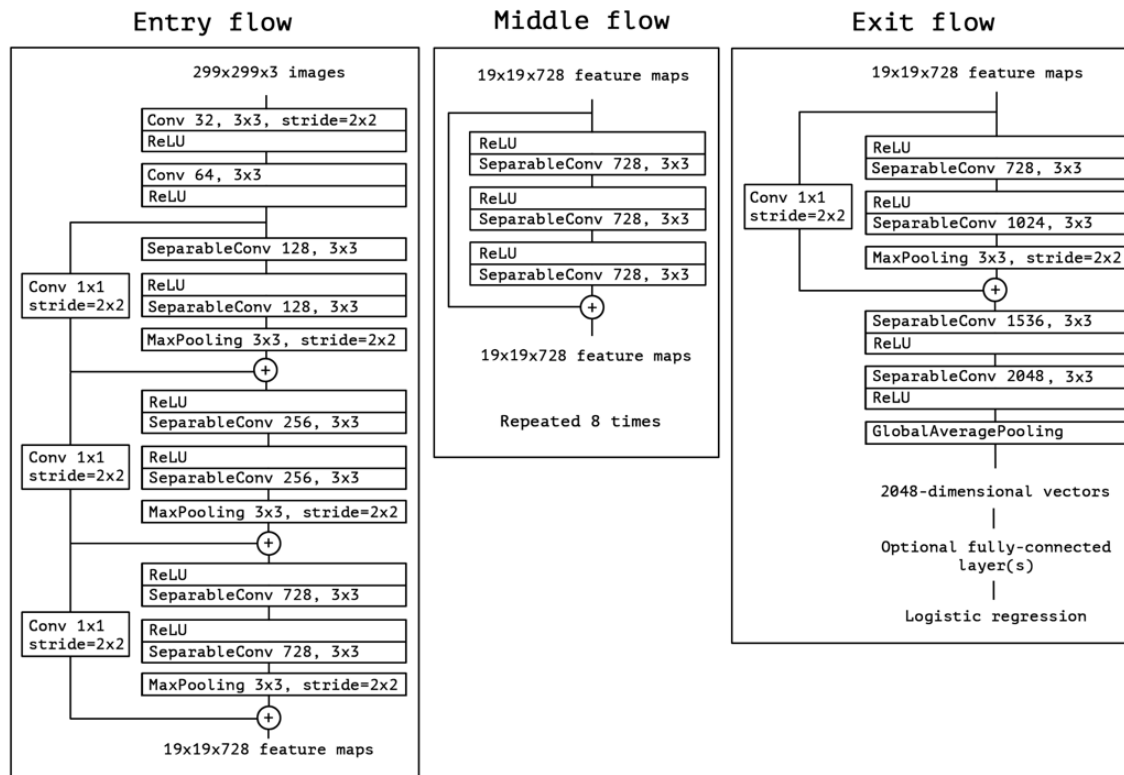
- Convolve each channel separately with a $n \times n$ filter to produce 1 channel output per input channel
- Use a 1×1 filter (called pointwise filter here) across the channels
- This is similar to the (extreme) inception module before, but in the reverse order
- Implementation in pytorch:

```
import torch.nn as nn

class depthwise_separable_conv(nn.Module):
    def __init__(self, nin, nout, kernel_size = 3, padding = 1, bias=False):
        super(depthwise_separable_conv, self).__init__()
        self.depthwise = nn.Conv2d(nin, nin, kernel_size=kernel_size, padding=padding, groups=nin, bias=bias)
        self.pointwise = nn.Conv2d(nin, nout, kernel_size=1, bias=bias)

    def forward(self, x):
        out = self.depthwise(x)
        out = self.pointwise(out)
        return out
```

Xception: Network with Depthwise Seperable Convolutions



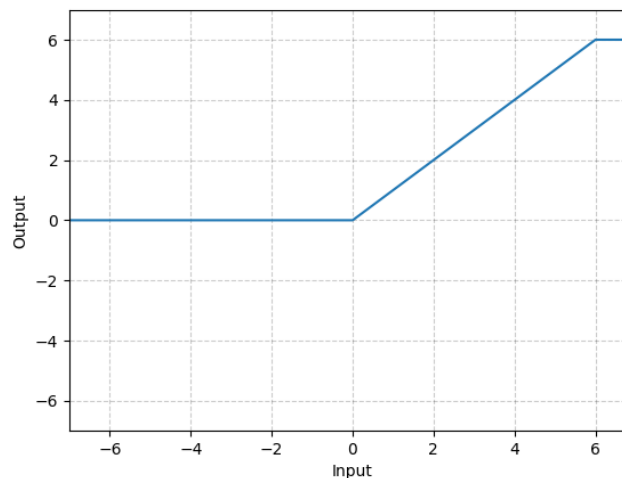
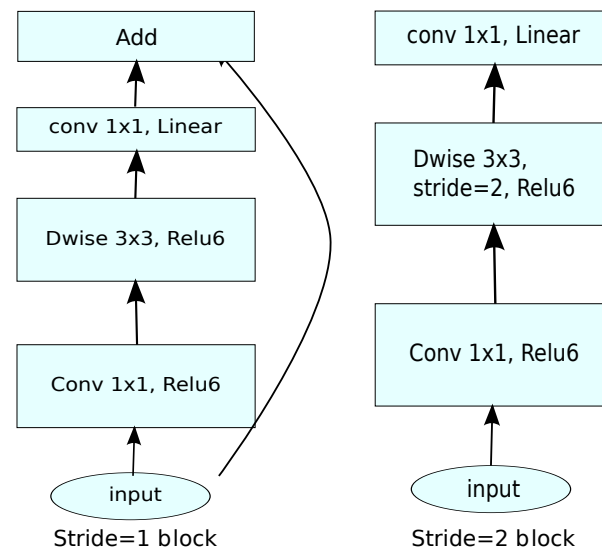
MobileNet V2

Lightweight Design

Bottleneck blocks with Depthwise separable convolutions

Relu 6 activation

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-



EfficientNet

EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks

Mingxing Tan¹ Quoc V. Le¹

Idea:

- Networks can be scaled in width, depth or image resolution
- Most approaches have only looked at one scaling
- EfficientNet proposes a *compound scaling method*:

$$\text{depth: } d = \alpha^\phi$$

$$\text{width: } w = \beta^\phi$$

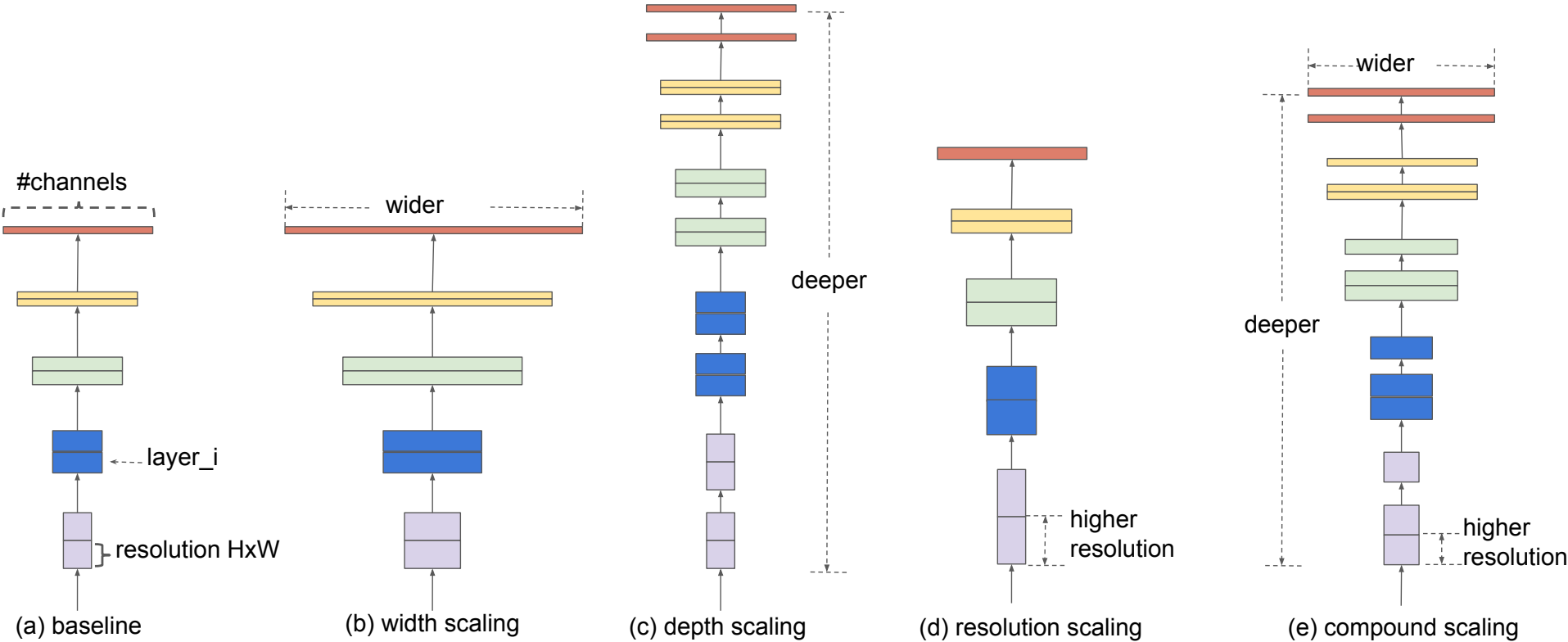
$$\text{resolution: } r = \gamma^\phi$$

$$\text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

- Parameters are found using a small grid search

Model scaling



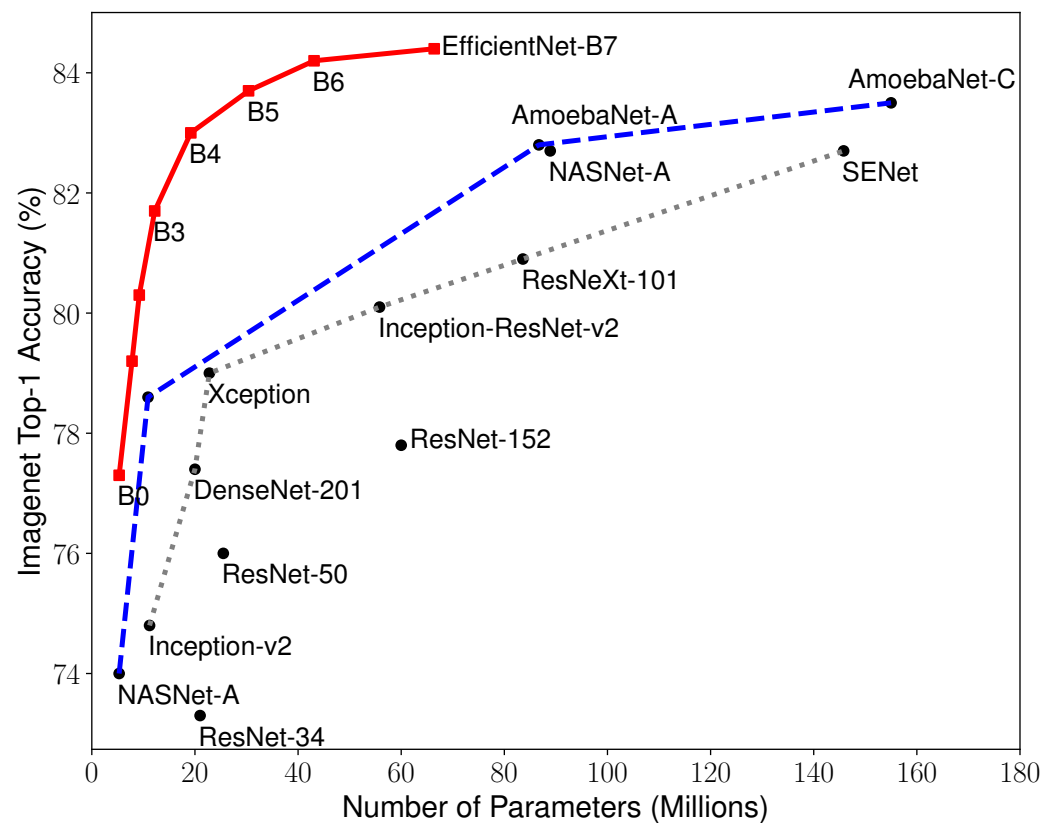
Efficient Net Architecture: EfficientNet-B0 baseline

Stage i	Operator $\hat{\mathcal{F}}_i$	Resolution $\hat{H}_i \times \hat{W}_i$	#Channels $\hat{C}_i$	#Layers $\hat{L}_i$
1	Conv3x3	224×224	32	1
2	MBConv1, k3x3	112×112	16	1
3	MBConv6, k3x3	112×112	24	2
4	MBConv6, k5x5	56×56	40	2
5	MBConv6, k3x3	28×28	80	3
6	MBConv6, k5x5	14×14	112	3
7	MBConv6, k5x5	14×14	192	4
8	MBConv6, k3x3	7×7	320	1
9	Conv1x1 & Pooling & FC	7×7	1280	1

MBConv = Mobile Inverted Bottleneck

Model	Top-1 Acc.	Top-5 Acc.	#Params	Ratio-to-EfficientNet	#FLOPs	Ratio-to-EfficientNet
EfficientNet-B0	77.1%	93.3%	5.3M	1x	0.39B	1x
ResNet-50 (He et al., 2016)	76.0%	93.0%	26M	4.9x	4.1B	11x
DenseNet-169 (Huang et al., 2017)	76.2%	93.2%	14M	2.6x	3.5B	8.9x
EfficientNet-B1	79.1%	94.4%	7.8M	1x	0.70B	1x
ResNet-152 (He et al., 2016)	77.8%	93.8%	60M	7.6x	11B	16x
DenseNet-264 (Huang et al., 2017)	77.9%	93.9%	34M	4.3x	6.0B	8.6x
Inception-v3 (Szegedy et al., 2016)	78.8%	94.4%	24M	3.0x	5.7B	8.1x
Xception (Chollet, 2017)	79.0%	94.5%	23M	3.0x	8.4B	12x
EfficientNet-B2	80.1%	94.9%	9.2M	1x	1.0B	1x
Inception-v4 (Szegedy et al., 2017)	80.0%	95.0%	48M	5.2x	13B	13x
Inception-resnet-v2 (Szegedy et al., 2017)	80.1%	95.1%	56M	6.1x	13B	13x
EfficientNet-B3	81.6%	95.7%	12M	1x	1.8B	1x
ResNeXt-101 (Xie et al., 2017)	80.9%	95.6%	84M	7.0x	32B	18x
PolyNet (Zhang et al., 2017)	81.3%	95.8%	92M	7.7x	35B	19x
EfficientNet-B4	82.9%	96.4%	19M	1x	4.2B	1x
SENet (Hu et al., 2018)	82.7%	96.2%	146M	7.7x	42B	10x
NASNet-A (Zoph et al., 2018)	82.7%	96.2%	89M	4.7x	24B	5.7x
AmoebaNet-A (Real et al., 2019)	82.8%	96.1%	87M	4.6x	23B	5.5x
PNASNet (Liu et al., 2018)	82.9%	96.2%	86M	4.5x	23B	6.0x
EfficientNet-B5	83.6%	96.7%	30M	1x	9.9B	1x
AmoebaNet-C (Cubuk et al., 2019)	83.5%	96.5%	155M	5.2x	41B	4.1x
EfficientNet-B6	84.0%	96.8%	43M	1x	19B	1x
EfficientNet-B7	84.3%	97.0%	66M	1x	37B	1x
GPipe (Huang et al., 2018)	84.3%	97.0%	557M	8.4x	-	-

Efficient Net vs. other architectures



Convolutional Neural Networks in Practice: Fine-tuning existing models

The discussed models were trained on the **ImageNet dataset**, which contains more than **14 million labeled images**!

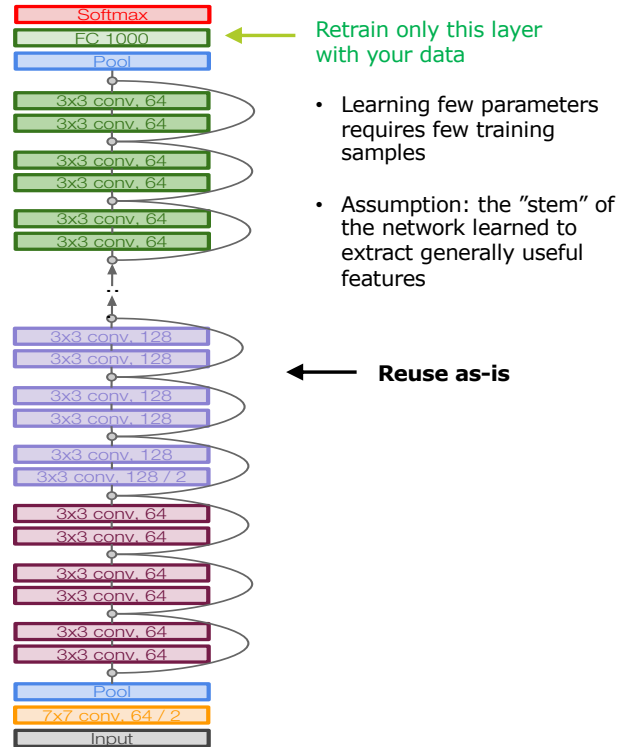
In practice, you rarely have access to labeled datasets of this size.

What to do?

- Fine-tuning an existing model

Available models

Model	Size	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth
Xception	88 MB	0.790	0.945	22,910,480	126
VGG16	528 MB	0.713	0.901	138,357,544	23
VGG19	549 MB	0.713	0.900	143,667,240	26
ResNet50	98 MB	0.749	0.921	25,636,712	-
ResNet101	171 MB	0.764	0.928	44,707,176	-
ResNet152	232 MB	0.766	0.931	60,419,944	-
ResNet50V2	98 MB	0.760	0.930	25,613,800	-
ResNet101V2	171 MB	0.772	0.938	44,675,560	-
ResNet152V2	232 MB	0.780	0.942	60,380,648	-
InceptionV3	92 MB	0.779	0.937	23,851,784	159
InceptionResNetV2	215 MB	0.803	0.953	55,873,736	572
MobileNet	16 MB	0.704	0.895	4,253,864	88
MobileNetV2	14 MB	0.713	0.901	3,538,984	88
DenseNet121	33 MB	0.750	0.923	8,062,504	121
DenseNet169	57 MB	0.762	0.932	14,307,880	169
DenseNet201	80 MB	0.773	0.936	20,242,984	201
NASNetMobile	23 MB	0.744	0.919	5,326,716	-
NASNetLarge	343 MB	0.825	0.960	88,949,818	-
EfficientNetB0	29 MB	-	-	5,330,571	-
EfficientNetB1	31 MB	-	-	7,856,239	-
EfficientNetB2	36 MB	-	-	9,177,569	-
EfficientNetB3	48 MB	-	-	12,320,535	-
EfficientNetB4	75 MB	-	-	19,466,823	-
EfficientNetB5	118 MB	-	-	30,562,527	-
EfficientNetB6	166 MB	-	-	43,265,143	-
EfficientNetB7	256 MB	-	-	66,658,687	-



Transfer Learning

- A trained model for image classification on a large data set is expected to have learned feature extraction that is relevant to any image classification tasks
- A typical architecture consists of convolutional layers, followed by a classification head with one or more fully connected layers
- If we just use the convolutional layers, they will do the **feature extraction** and we can attach a new head for the classification (most likely for a different number of classes)
- The convolutional layers should be **frozen**, so that their weights are not updated
- They can be unfrozen and updated at the end using a very small learning rate, this is called **fine-tuning**

Caveats

- Base networks might have been trained on different image sizes
- They often work for a range of sizes if the classification head is excluded, but not for arbitrary sizes, for example InceptionV3 works for sizes > 75 , others for sizes as small as 32
- If the input image are much larger (or much smaller) than they were trained for (for example 4k), they might not perform as well, and the output features will be too large
- If you have a completely different application domain, you should not use transfer learning!

Convolutional Neural Networks in Practice: Augmenting your Dataset

The discussed models were trained on the **ImageNet dataset**, which contains more than **14 million labeled images!**

In practice, you rarely have access to labeled datasets of this size.

What to do?

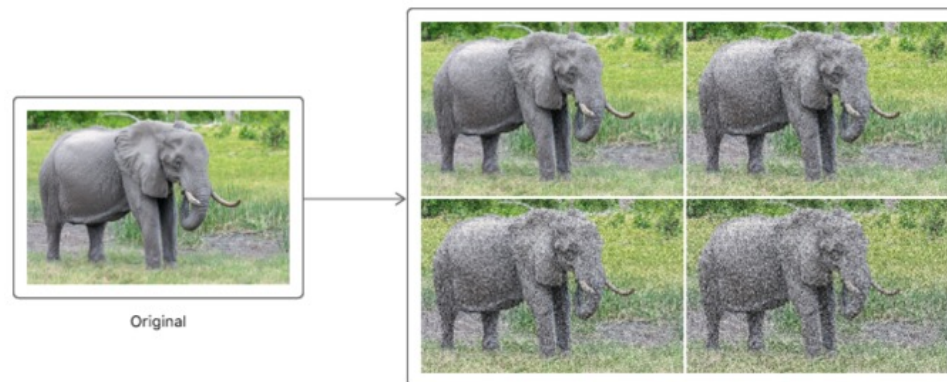
- Fine-tuning an existing model and/or
- Augmenting your Dataset

As long as a human can classify the image, the CNN should be able to do too!

It also makes the CNN more robust!



Source: <https://medium.com/analytics-vidhya/data-augmentation-is-it-really-necessary-b3cb12ab3c3f>



Source: <https://developer.apple.com/documentation/coreml/mlimageclassifier/imageaugmentationoptions/noise>

Image Augmentation

- Applied only on the training set, not validation or test
- Implemented as part of the model (less space, more training time) or by generating an augmented version of the data set
- Typically, with parameters giving the amount of augmentation, adds to the hyper parameters
- Geometric (Rotation, Scaling, Flip, Crop, etc.)
- Color (Contrast, Brightness, Color Adjustments, etc.)



Auto Augmentation

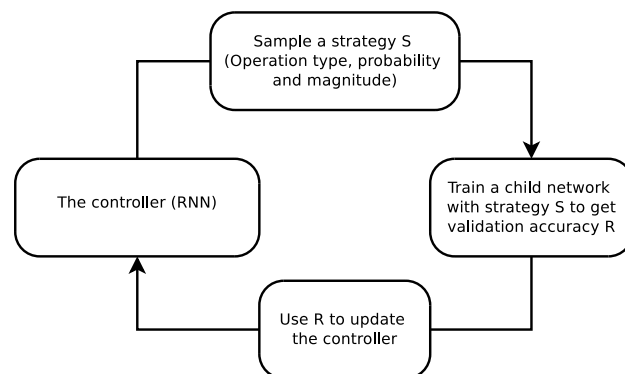
AutoAugment: Learning Augmentation Strategies from Data

Ekin D. Cubuk *, Barret Zoph*, Dandelion Mané, Vijay Vasudevan, Quoc V. Le
Google Brain

It can be difficult to find the optimal methods and parameters for the augmentation:

AutoAugment approach:

- Automatically search for data augmentation policies



	Original	Sub-policy 1	Sub-policy 2	Sub-policy 3	Sub-policy 4	Sub-policy 5
Batch 1						
Batch 2						
Batch 3						
		Equalize, 0.4, 4 Rotate, 0.8, 8	Solarize, 0.6, 3 Equalize, 0.6, 7	Posterize, 0.8, 5 Equalize, 1.0, 2	Rotate, 0.2, 3 Solarize, 0.6, 8	Equalize, 0.6, 8 Posterize, 0.4, 6

- Best policies on ImageNet favor Color Augmentation